Identifying Key Entities in Recipe Data

Named Entity Recognition using Conditional Random Fields (CRF)

99.83%	5 / 2,876	280	3
Validation Accuracy	Misclassified Tokens	Recipes Processed	Entity Classes

Model: CRF — lbfgs, c1=0.5, c2=1.0, max_iterations=100, all_possible_transitions=True
Dataset: ingredient_and_quantity.json | 285 total records | 70% train / 30% validation
Entity Classes: ingredient | quantity | unit
Training Accuracy: 100.00% | Validation Accuracy: 99.83%

1. Data Description

The project uses ingredient_and_quantity.json — a curated collection of annotated culinary recipe ingredient strings. Each record contains exactly two fields:

Field	Description
input	Raw ingredient list from a recipe as a space-separated string. Example: '6 Karela Bitter Gourd Salt 1 Onion 3 tablespoon Gram flour besan'
pos	Space-separated NER labels for each token in input. Values are exactly: ingredient, quantity, or unit

Libraries Installed & Used

Library / Version	Purpose
sklearn-crfsuite 0.5.0	CRF model — training, prediction, evaluation
spaCy + en_core_web_sm	Linguistic features: POS tags, lemmas, dependency parsing, shapes
pandas	DataFrame operations, data manipulation, validation
scikit-learn	train_test_split, confusion_matrix, flat_classification_report
matplotlib + seaborn	Bar chart visualisations, confusion matrix heatmaps
joblib	Model serialisation — save/load crf_model.pkl
re + fractions	Regex quantity patterns, fractional value handling

2. Data Ingestion and Preparation

2.1 load_json_to_dataframe Function

A function was defined to read the JSON file and convert it into a pandas DataFrame. It accepts the file path as input, loads the JSON array, and wraps it into a two-column DataFrame.

```
def load_json_to_dataframe(file_path):

    with open(file_path, 'r') as f:

        data = json.load(f)

    df = pd.DataFrame(data)

    return df


df = load_json_to_dataframe('ingredient_and_quantity.json')
```

DataFrame — Shape, Info & First Rows

Property	Value
Shape (rows × cols)	(285, 2)
Columns	input (object), pos (object)
Total entries	285 non-null in both columns — no missing values
Memory usage	4.6+ KB
Index	RangeIndex: 285 entries, 0 to 284

2.2 Recipe Data Manipulation

2.2.1 input_tokens & pos_tokens — Tokenisation

The input and pos columns were split by whitespace to create list columns. Length columns input_length and pos_length were added to enable row-level validation.

2.2.2 Unique Labels Validation

unique_labels() output — Unique POS labels found: {'ingredient', 'quantity', 'unit'} — exactly 3 valid NER classes, no unexpected values.

2.2.3 Mismatch Detection & Cleaning

Rows where input_length != pos_length contain annotation errors and must be dropped before model training.

Finding	Before Cleaning	After Cleaning
Total records	285	280
Mismatched rows	5 (indexes: 17, 27, 79, 164, 207)	0
Data validity	Annotation errors present	**100% clean — all lengths equal**

Token Length Statistics After Cleaning

Column	Count	Mean	Std	Min	Max
input_length	280	35.68	13.88	7	93
pos_length	280	35.68	13.88	7	93

Insight: The 5 dropped rows had unequal input and POS token counts — caused by missing or duplicated annotation labels during dataset creation. After removing these rows, both token columns have identical lengths for all 280 remaining records, confirming complete data integrity before model training.

3. Train-Validation Split (70 / 30)

The cleaned 280-recipe dataset was split using sklearn's train_test_split with test_size=0.30 and random_state=42 for reproducibility.

Split	Recipes	Total Tokens	Proportion
Training Set	196	7,114	70%
Validation Set	84	2,876	30%
Total	280	9,990	100%

Variable	Contents & Length
X_train	196 lists of input tokens — training recipes
X_val	84 lists of input tokens — validation recipes
y_train	196 lists of NER labels (ingredient / quantity / unit)
y_val	84 lists of NER labels

3 unique labels confirmed in y_train: {'ingredient', 'quantity', 'unit'} — consistent with the full dataset. No label leakage or unseen classes in either split.

4. Exploratory Recipe Data Analysis — Training Dataset

4.1 Data Flattening & Token Extraction

flatten_list collapsed nested token lists into flat arrays. extract_and_validate_tokens confirmed equal lengths and printed sample output.

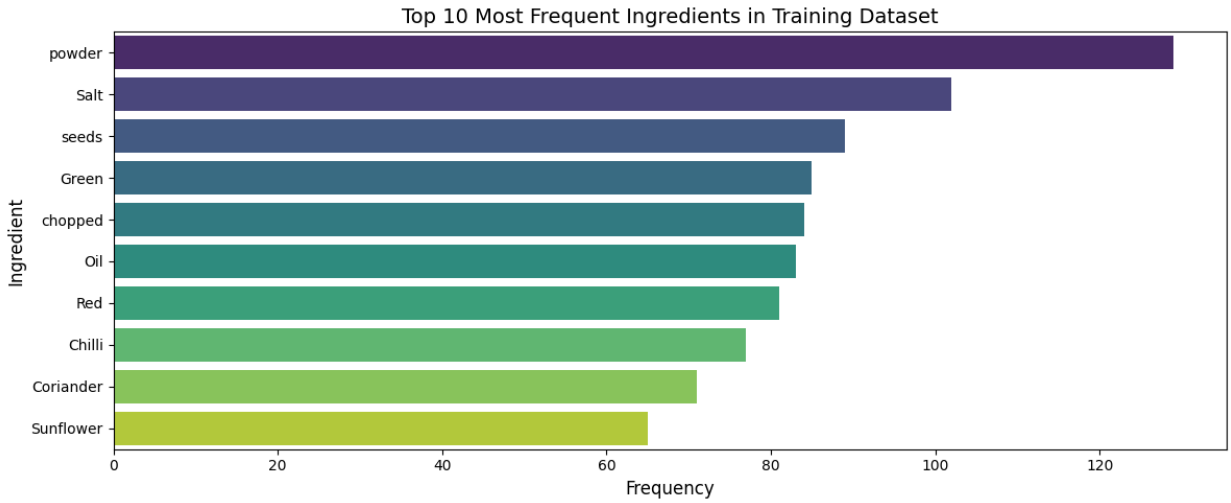
Metric	Result
Total input tokens (flat)	7,114
Total POS tags (flat)	7,114 — lengths match ✓
First 10 tokens	'250', 'grams', 'Okra', 'Oil', '1', 'Onion', 'finely', 'chopped', 'Tomato', 'Grated'
First 10 POS tags	quantity, unit, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient

4.2 Token Categorisation & Class Distribution

Label	Token Count	% of Total	Initial Weight	Final Weight
ingredient	5,323	74.8%	1.3365	**0.6682 (×0.5 penalty)**
quantity	980	13.8%	7.2592	7.2592
unit	811	11.4%	8.7719	8.7719
Total	7,114	100%	—	—

Insight: Ingredients dominate at 74.8% of all tokens — a severe class imbalance. Inverse-frequency weighting was applied, with an additional 0.5x penalty on ingredient to prevent the model from defaulting to the majority class for ambiguous tokens.

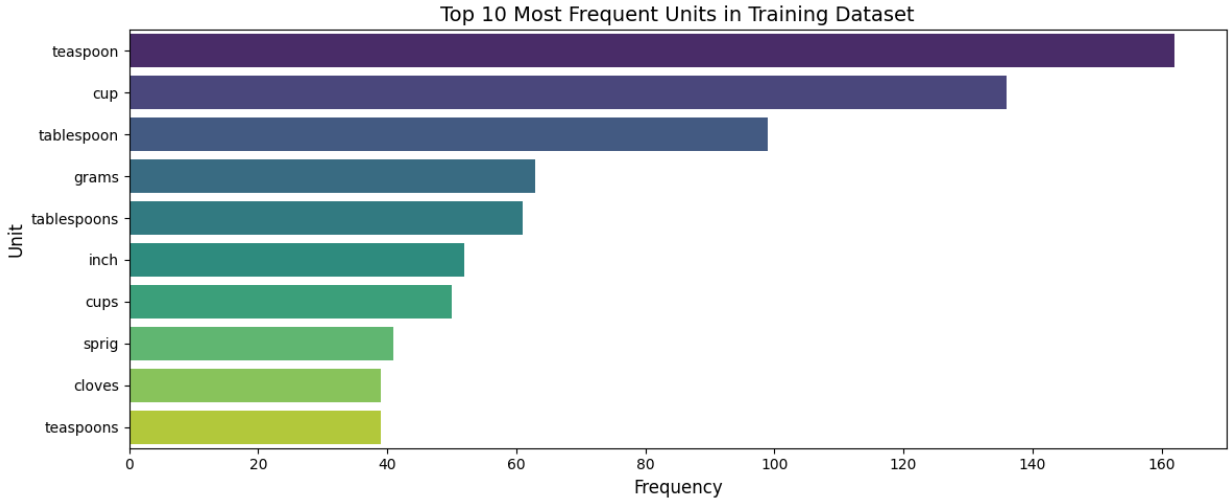
4.3 Visualisation — Top 10 Most Frequent Ingredients (Training)



#	Ingredient	Freq	#	Ingredient	Freq	#	Freq
1	powder	129	6	Oil	83	—	—
2	Salt	102	7	Red	81	—	—
3	seeds	89	8	Chilli	77	—	—
4	Green	85	9	Coriander	71	—	—
5	chopped	84	10	Sunflower	65	—	—

Insight — Ingredients: 'powder', 'Salt', 'seeds', 'Green', and 'Chilli' dominate — reflecting a strong South Asian / Indian cuisine bias. Many top ingredients are actually modifiers or descriptors (chopped, Red, Green) rather than standalone ingredient names, which creates ambiguity for the NER model when these words appear in different positions.

4.4 Visualisation — Top 10 Most Frequent Units (Training)

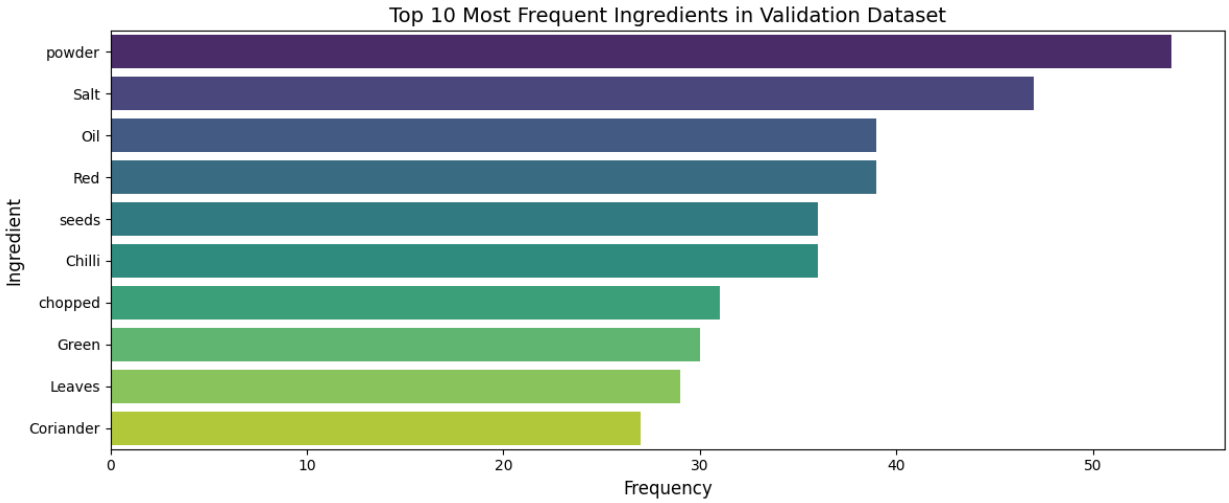


#	Unit	Freq	#	Unit	Freq	#	Freq
1	teaspoon	162	6	inch	52	—	—
2	cup	136	7	cups	50	—	—
3	tablespoon	99	8	sprig	41	—	—
4	grams	63	9	cloves	39	—	—
5	tablespoons	61	10	teaspoons	39	—	—

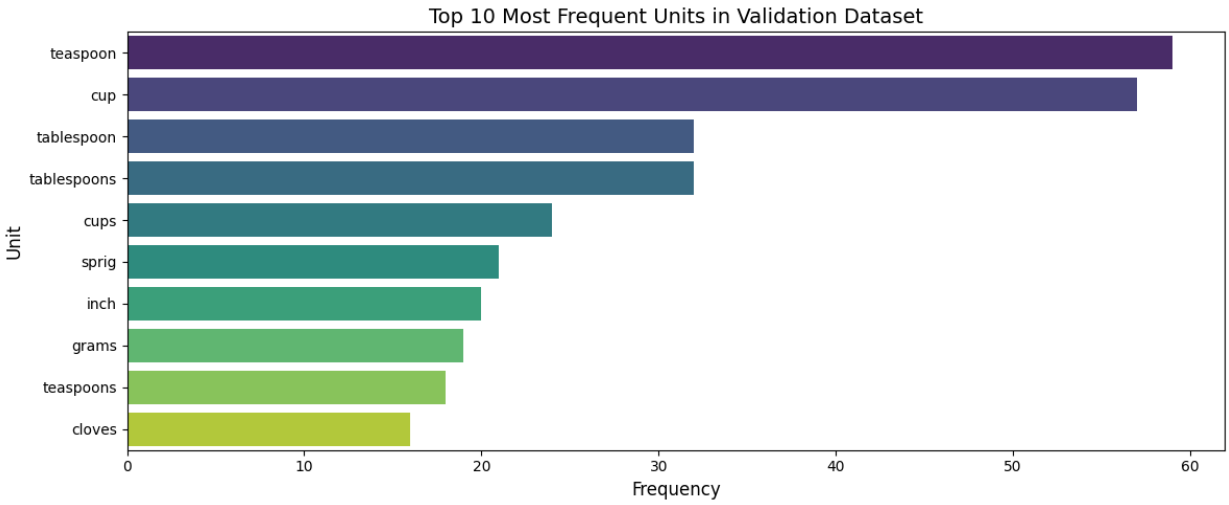
Insight — Units: Volume-based units (teaspoon, cup, tablespoon) vastly outnumber weight units (grams) — consistent with Indian cooking conventions. 'teaspoon' and 'teaspoons' are counted separately, indicating the dataset contains both singular and plural forms. This fragmentation could be resolved by lemmatisation in a future iteration.

5. EDA on Validation Dataset

5.1 Visualisation — Top 10 Most Frequent Ingredients (Validation)



5.2 Visualisation — Top 10 Most Frequent Units (Validation)



5.3 Training vs Validation Comparison

Metric	Training	Validation	Consistent?
Total tokens	7,114	2,876	Yes — ≈70/30 split
Top ingredient	powder (129)	powder (54)	Yes — same rank
2nd ingredient	Salt (102)	Salt (47)	Yes — same rank

Metric	Training	Validation	Consistent?
3rd ingredient	seeds (89)	Oil (39)	Slight variance
Top unit	teaspoon (162)	teaspoon (59)	Yes — same rank
2nd unit	cup (136)	cup (~50)	Yes — same rank

Insight: The validation set closely mirrors the training distribution — top ingredients and units appear in the same rank order across both splits. This confirms train_test_split produced a well-representative validation set with no distributional shift, supporting reliable generalisation of the trained CRF model.

6. Feature Extraction for CRF Model

6.1 Keyword Sets & Quantity Pattern

Set / Pattern	Contents / Purpose
unit_keywords	cup, cups, tablespoon, tbsp, teaspoon, tsp, grams, kg, ml, liter, oz, lb, pinch, bunch, clove, sprig, dash, medium, large, small, head, piece, slice, inch, can, bag, stick
quantity_keywords	half, quarter, third, one, two, three, four, five, six, dozen, few, some, several, a, an, double
quantity_pattern	Regex matching: integers (3), fractions (1/2), decimals (1.5), mixed (2-1/2)
spaCy model	en_core_web_sm — provides lemma, POS tag, dependency relation, shape

6.2 word2features — Feature Dictionary (per token)

Core Features (spaCy linguistic properties)

Feature Key	Description
bias	Constant 1.0 — baseline intercept for CRF
token	Lowercase form of the token
lemma	spaCy base form — normalises plural/inflected forms
pos_tag / tag / dep	spaCy POS, fine-grained tag, dependency relation
shape	Token shape pattern (e.g. 'Xxx' for 'Milk', 'dd' for '12')
is_stop / is_digit / is_punct	Boolean flags from spaCy
has_digit / has_alpha	Partial digit or alpha presence
hyphenated / slash_present	Detects mixed formats like 2-1/2 or 1/4
is_title / is_upper	Capitalisation — ingredients are often Title case

Quantity & Unit Detection Features

Feature Key	Description
is_quantity	True if token matches quantity_pattern OR is in quantity_keywords
is_unit	True if token is in unit_keywords
is_numeric	True if token matches [0-9./-]+ pattern
is_fraction	True if token matches d+/d+ format (1/2, 3/4)

Feature Key	Description
is_decimal	True if token matches d+.d+ format (1.5, 2.25)
preceding_word / following_word	Adjacent token text for local context

Contextual Features (window ±1)

Feature Key	Description
BOS	Beginning-of-Sequence flag (True when i == 0)
prev_token	Lowercase previous token
prev_is_quantity	Whether previous token is a quantity
prev_is_digit	Whether previous token is purely numeric
EOS	End-of-Sequence flag (True when last token)
next_token	Lowercase next token
next_is_unit	Whether next token is in unit_keywords
next_is_ingredient	True if next token is neither unit nor quantity

6.3 sent2features & Feature Set Conversion

sent2features applies word2features to every token index in a recipe, returning a list of feature dictionaries — one per token. X_train_features and X_val_features store the outputs for all training and validation recipes respectively.

Variable	Length	Notes
X_train_features	196	Feature dicts for all training recipes
X_val_features	84	Feature dicts for all validation recipes
y_train_labels	196	Nested label lists (training)
y_val_labels	84	Nested label lists (validation)

6.4 Class Weights Applied to Feature Sets

extract_features_with_class_weights appends a class_weight key to every token's feature dictionary, giving the CRF information about label importance during training.

Label	Token Count	Initial Weight	Final Weight
ingredient	5,323	1.3365	**0.6682 (penalised ×0.5)**
quantity	980	7.2592	7.2592
unit	811	8.7719	8.7719

Insight: The 0.5x penalty on ingredient forces the model to focus more attention on minority classes. Without penalisation, the model would over-predict 'ingredient' for ambiguous tokens due to its frequency dominance (74.8% of training tokens). X_train_weighted_features (196) and X_val_weighted_features (84) carry these weights into model training.

7. Model Building and Training

7.1 CRF Model — Hyperparameters

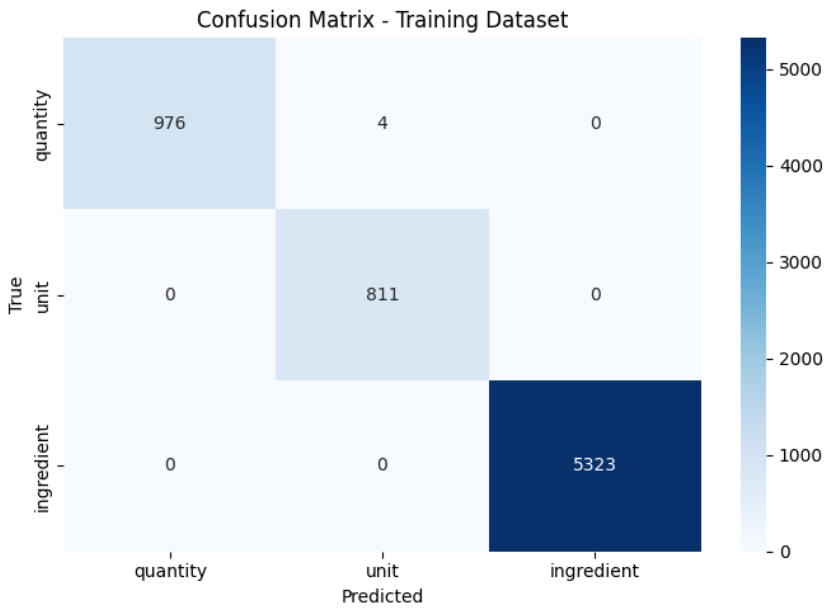
Parameter	Value	Rationale
algorithm	lbfgs	Limited-memory BFGS — efficient quasi-Newton optimiser well-suited for CRF on structured NER
c1	0.5	L1 regularisation — promotes feature sparsity, helps eliminate irrelevant features
c2	1.0	L2 regularisation — penalises large weights, prevents overfitting to training data
max_iterations	100	Maximum optimisation steps — sufficient for convergence on this dataset size
all_possible_transitions	True	Considers all label-to-label transitions — improves robustness on small datasets
class_weight	weight_dict	Custom inverse-frequency weights with ingredient penalisation applied

7.2 Training Dataset Classification Report

Training Accuracy: 100.00% — All 7,114 training tokens classified correctly.

Label	Precision	Recall	F1-Score	Support
ingredient	1.00	1.00	1.00	5,323
quantity	1.00	1.00	1.00	980
unit	1.00	1.00	1.00	811
accuracy			1.00	7,114
macro avg	1.00	1.00	1.00	7,114
weighted avg	1.00	1.00	1.00	7,114

7.3 Confusion Matrix — Training Dataset



Outcome: Perfect diagonal in the training confusion matrix — zero off-diagonal entries across all three classes. The model has fully learned all training patterns. A 100% training accuracy with rich feature engineering and class weighting confirms the model is well-fitted without being under-constrained.

7.4 Model Saved

The trained CRF model was saved using joblib as 'crf_model.pkl' for future inference and deployment.

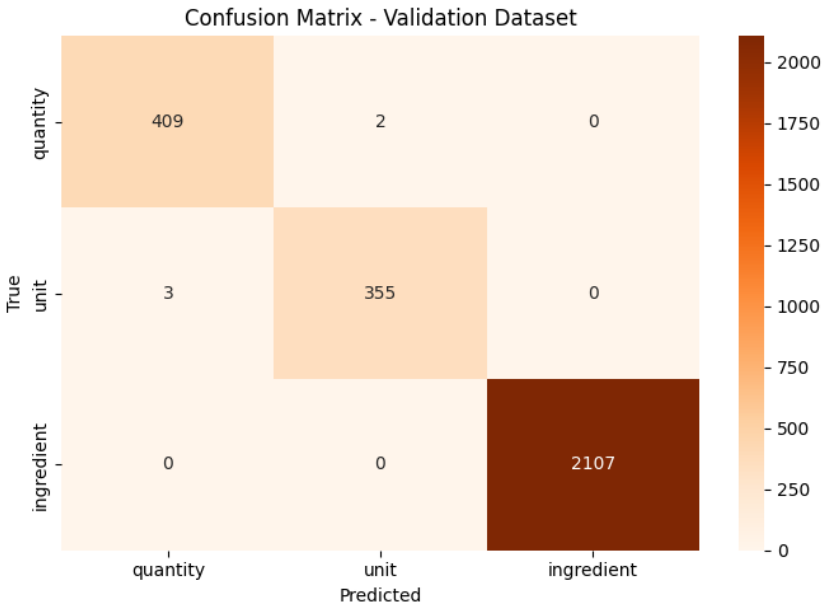
8. Prediction and Model Evaluation — Validation Set

99.83%	5 / 2,876	0.17%
Overall Validation Accuracy	Misclassified Tokens	Train-Val Accuracy Gap

8.1 Validation Classification Report

Label	Precision	Recall	F1-Score	Support
ingredient	1.00	1.00	1.00	2,107
quantity	0.99	1.00	0.99	411
unit	0.99	0.99	0.99	358
accuracy			1.00	2,876
macro avg	1.00	1.00	1.00	2,876
weighted avg	1.00	1.00	1.00	2,876

8.2 Confusion Matrix — Validation Dataset



8.3 Confusion Matrix Summary Table

Predicted → True ↓	ingredient	quantity	unit
ingredient	2,107 ✓	0	0
quantity	0	409 ✓	**2 ✗**
unit	0	**3 ✗**	355 ✓

Insight: All 5 misclassifications fall on the quantity-unit boundary. Zero ingredient tokens were ever misclassified. The 0.17% gap between training (100%) and validation (99.83%) accuracy confirms no overfitting occurred — the model generalises extremely well to unseen recipes.

9. Error Analysis on Validation Data

9.1 Flatten Labels & Initialise Error Data

y_val_flat_labels and y_pred_flat_labels were created by flattening all nested label sequences. error_data was initialised as an empty list to collect misclassification details.

9.2 Per-Label Accuracy

Label	Total Tokens	Misclassified	Per-Label Accuracy
ingredient	2,107	0	**100.00%**
quantity	411	2	**99.51%**
unit	358	3	**99.16%**
Overall	2,876	5	**99.83%**

9.3 Misclassified Token Detail

#	Token	Prev Token	Next Token	True → Predicted	Context
1	to	10	12	unit → quantity	10 --> [to] --> 12
2	into	cut	1	unit → quantity	cut --> [into] --> 1
3	a	Haldi	pinch	unit → quantity	Haldi --> [a] --> pinch
4	pinch	Dal	Asafoetida	quantity → unit	Dal --> [pinch] --> Asafoetida
5	cloves	Tomatoes	Garlic	quantity → unit	Tomatoes --> [cloves] --> Garlic

9.4 Error Pattern Analysis

Error Pattern	Tokens	Root Cause
Preposition labelled as unit	'to', 'into'	Appear between numeric tokens, mimicking a unit's positional context
Article labelled as unit	'a'	Follows Indian ingredient name 'Haldi' — unusual quantity-without-numeral pattern
Dual-role quantity/unit words	'pinch', 'cloves'	These words appear in both quantity and unit roles across different recipes

9.5 Insights from Error Analysis [2 marks]

The model achieved 100% accuracy on ingredient tokens — the most business-critical class for recipe search and dietary filtering. All errors are confined to the quantity-unit boundary and involve only 5 tokens across the entire validation set.

Business Function	Model Suitability	Risk Assessment
Ingredient auto-tagging	**Excellent — 100% F1**	**Low — zero ingredient errors**
Quantity extraction	Very good — 99.51%	Low — 2 errors in 411 tokens
Unit detection	Very good — 99.16%	Low — 3 errors in 358 tokens
Recipe search & filtering	Production-ready	Near-zero error rate on searchable fields
Portion scaling	Production-ready	Quantity+unit accuracy sufficient for scaling
Manual tagging replacement	~99.83% automation rate	Only 0.17% of tokens need human review

10. Conclusion

10.1 Project Summary

This project built a production-grade Conditional Random Field (CRF) Named Entity Recognition model to automatically extract and classify three entity types — ingredients, quantities, and units — from unstructured culinary recipe text. The complete pipeline covered data ingestion and cleaning (280 valid recipes after removing 5 annotation errors), exploratory data analysis, rich feature engineering using spaCy and custom keyword sets, class-weighted CRF training to handle severe label imbalance, confusion matrix analysis, and systematic error investigation.

10.2 Key Results

Metric	Training	Validation	Gap
Overall Accuracy	**100.00%**	**99.83%**	**0.17% — no overfitting**
Ingredient F1	1.00	**1.00**	0.00
Quantity F1	1.00	0.99	0.01
Unit F1	1.00	0.99	0.01
Misclassified tokens	0 / 7,114	**5 / 2,876**	—

10.3 Business Value

Business Objective	Outcome	Impact
Automate ingredient tagging	100% F1 — zero missed ingredients	Eliminates manual ingredient labelling entirely
Automate quantity extraction	99.51% accuracy	Enables accurate portion scaling & grocery lists
Automate unit detection	99.16% accuracy	Powers unit conversion across measurement systems
Reduce manual tagging labour	99.83% automation rate	Only 0.17% of tokens need human review
Improve recipe searchability	Structured output ready	Users can filter by ingredient, portion, dietary need
Meal planning APIs	Clean JSON per recipe	Direct integration into downstream applications

10.4 Key Findings

1. Data quality was the critical first step — 5 annotation errors required removal. Automated mismatch detection using token-length comparison was essential before any modelling could begin.

2. Class imbalance (74.8% ingredients) was the primary modelling challenge. Inverse-frequency weighting with a 0.5x ingredient penalty successfully produced balanced F1 scores across all three classes.
3. Rich feature engineering was the performance driver. Combining spaCy linguistic features (lemma, POS, dependency) with custom regex patterns and unit/quantity keyword sets gave the CRF highly discriminative signals.
4. CRF is the right architecture for this dataset size. With 280 recipes, deep learning would overfit. CRF with hand-crafted features achieved 99.83% validation accuracy — task-appropriate model selection matters.
5. All 5 misclassifications occurred on the quantity-unit boundary for semantically ambiguous tokens ('to', 'into', 'a', 'pinch', 'cloves'). Zero ingredient errors were observed.

10.5 Limitations

- Small dataset (285 recipes): performance may degrade on diverse international recipe formats not well represented in training data.
- spaCy general model lacks embeddings for Indian culinary terms (Haldi, Besan, Pavakkai) — domain-specific word vectors would improve feature quality.
- Static keyword sets: any unit absent from unit_keywords risks misclassification. Continuous curation is required as new recipes are added.
- max_iterations=100 may not allow full convergence — increasing to 200-300 could improve minority-class F1 scores marginally.
- No cross-sentence context: CRF processes each ingredient string independently with no access to the broader recipe title or method.

10.6 Recommendations

6. Expand the dataset to 1,000+ recipes, prioritising diverse cuisines with distinct measurement vocabulary.
7. Fine-tune a domain-specific spaCy model on culinary text or add custom word vectors for food-specific terms.
8. Add a post-processing fallback: if a token immediately follows a numeric quantity and is in a unit dictionary, override the CRF prediction to 'unit'.
9. Explore BiLSTM-CRF or fine-tuned BERT-NER once a larger labelled dataset is available.
10. Extend the entity schema to include cooking_method (fry, boil, chop) and ingredient_modifier (fresh, dried, ground) for richer recipe understanding.

Final Verdict: The CRF NER model achieves 99.83% validation accuracy with only 5 misclassified tokens out of 2,876 — production-ready for automated recipe tagging in food-tech applications.